

Computer Programming (Unit – III) - Syllabus

Decision making statements – branching and looping – arrays – multidimensional arrays – Functions – Recursion – Passing array to functions. Storage classes – Strings – String library functions.

Part – A (2 Marks)**1. Compare while and do-while statements. (Dec 2017)(May 2017)(May 2018)**

while	do-while
Condition is checked first then statement(s) is executed.	Statement(s) is executed at least once, thereafter condition is checked.
It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed.
No semicolon at the end of while. while(condition)	Semicolon at the end of while. while(condition);
If there is a single statement, brackets are not required.	Brackets are always required.
Variable in condition is initialized before the execution of loop.	variable may be initialized before or within the loop.
while loop is entry controlled loop.	do-while loop is exit controlled loop.
while(condition) { statement(s); }	do { statement(s); }while(condition);

2. List any five built in string functions. (May 2017)(Dec 2017)

Function	Description
strrev()	It reverses the given string

strcmp()	Compares the given string for the equality
strlen()	Returns the length of a string
strstr()	Returns a string representation of an object
strcpy()	Copies a string from source to destination

3. What is meant by looping ? (May 2014)

- A **Loop** executes the sequence of statements many times until the stated condition becomes false.
- A **loop** consists of two parts, a body of a **loop** and a control statement.

4. What is a user –defined datatype? (May 2014)

The data types that are defined by the user are called the derived datatype or user-defined derived data type.

These types include:

- Structure
- Union
- Enumeration
- Typedef defined DataType

5. Define recursion. (Jan 2015)

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function. Using recursive algorithm, certain problems can be solved quite easily.

General Form:

```
function ()  
{  
    function();  
}
```

Examples:

Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph, etc.

6. Let 'a' and 'b' be two strings "Rama" and "Ramu". what will be the output of strcmp(a,b)? (Jan 2015)

strcmp(a,b) – returns a non-zero value, which means both are not equal.

This function return values that are as follows –

- if Return value < 0 then it indicates str1 is less than str2.
- if Return value > 0 then it indicates str2 is less than str1.
- if Return value = 0 then it indicates str1 is equal to str2.

7. Write any four features of arrays. (May 2016)

- An array holds elements that have the same data type.
- Array elements are stored in subsequent memory locations.
- Two-dimensional array elements are stored row by row in subsequent memory locations.
- Array name represents the address of the starting element.

8. Give example of ternary operator. Give example. (May 2016)

The Ternary or conditional operator is kind of similar to the if-else statement as it does follow the same algorithm as of if-else statement but the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.

Syntax:

variable = Expression1 ? Expression2 : Expression3

int m = 5, n = 4;

**(m > n) ? printf("m is greater than n that is %d > %d", m, n)
: printf("n is greater than m that is %d > %d", n, m);**

9. Write an if then else to test it a number is below 100. (Nov 2016)

#include <stdio.h>

int main()

{

int number;

printf("Please enter a number: ");

scanf("%d", &number);

if (number < 100)

{

printf("Number is less than 100!\n");

}

else

```
{  
    printf("Number is greater than 100!\n");  
}  
return 0;  
}
```

output

Please enter a number: 10

Number is less than 100!

10. Let a be the string “Rama” what is the output of strlen(a); ? (Nov 2016)

```
#include <stdio.h>  
#include <string.h>  
int main()  
{  
    char a[100];  
    int length;  
  
    printf("Enter a string to calculate its length\n");  
    gets(a);  
  
    length = strlen(a);  
  
    printf("Length of the string = %d\n", length);  
  
    return 0;  
}
```

Output of a Program:

Enter a string to calculate its length

Rama

Length of the string = 4

11. How an actual argument differ from formal argument. (May 2018)

- Formal arguments are declared within the () whereas local variables are declared at beginning.
- Formal arguments are automatically initialized when value of actual arguments is passed.
- Where other local variables are assigned variables through the statements inside the function body.

12. Explain the differences between if and while statement. (Dec 2018)**IF statement:**

An if statement checks if an expression is true or false, and then runs the code inside the statement only if it is true.

```
if (x > y)
{
    // this will only happen once
}
```

While Statement:

A while statement is a loop. Basically, **it continues to execute** the code in the while statement for however long the expression is true.

```
while (x > y)
{
    // this will keep happening until the condition is false.
}
```

13. What are the difference between auto and static storage class. (Dec 2018)

- Auto is used for a local variable defined within a block or function.
- Static is used for both global and local variables. Each one has its use case within a C program.

14. What happens if an element outside the array boundary is referenced? (Sep 2020)

- The array index out of bounds error is a special case of the buffer overflow error.
- It occurs when the index used to address array items exceeds the allowed value.
- It's the area outside the array bounds which is being addressed, that's why this situation is considered a case of undefined behavior.

15. Illustrate the functionality of the 'strncmp' function in C with an example. (Sep 2020)

- **The strncmp() function** which is same as strcmp(), except that strncmp() comparison is limited to the number of characters specified during the function call.
- For example strncmp(str1, str2, 4) would compare only the first four characters of strings str1 and str2.

Part – B (11 marks)

1. Discuss the different storage classes by identifying its scope and lifetime of variables.(Jan 2015)(May 2018)(Dec 2018)

Storage Classes

- The **area or block of the C program** from where the variable can be accessed is known as **the scope of the variable**.
- Scope of a variable is also defined as the region over which **variable is visible** or valid.
- The area or scope of the variable **depends on its storage class**, that is, where and how it is declared.
- The storage class of a variable **tells the compiler**
 - Storage area of the variable
 - Initial value of the variable if not initialized
 - Scope of the variable
 - Life of the variable that is how long the variable would be active in the program.

There are four types of storage classes available in a C language. They are

1. **Automatic Variables**
2. **Static Variables**
3. **External Variables (Global Variables)**
4. **Register Variables**

The following table gives the detail information about various storage classes.

S. No.	Storage Specifier	Storage place	Initial / default value	Scope	Life
1	auto	CPU Memory	Garbage value	local	Within the function only.
2	extern	CPU memory	Zero	Global	Till the end of the main program. Variable definition might be anywhere in the C program
3	static	CPU memory	Zero	local	Retains the value of the variable between different function calls.
4	register	Register memory	Garbage value	local	Within the function

1. Automatic Variables

- **Variables declared inside a block** and local to block in which declared are said to be **Automatic** variables. These variables can be accessed by block in which they are declared. Another block cannot access its value.
- These variables created as new variable each time when function is called and destroyed automatically when the function is exited.
- One important feature of automatic variables is that their **value cannot be changed accidentally** by what happens in some other function in the program.

Example: ILLUSTRATION OF WORKING OF auto VARIABLES

```
void function1(void);
void function2(void);
main( )
{
    int m = 1000;
    function2();

    printf("%d\n",m);    /* Third output */
}
void function1(void)
{
    int m = 10;

    printf("%d\n",m);    /* First output */
}

void function2(void)
{
    int m = 100;
    function1();
    printf("%d\n",m);    /* Second output */
}
```

OUTPUT:

```
10
100
1000
```

2. Static Variables

- Static variable may be **either an internal type or an external type** depending on the place of declaration.
- Static variables declared within individual block. They are local to the block in which declared. It exists until the end of the program.
- Variable can be declared using the keyword **static**. Global and Local variable can be declared static.
- Static variables are **initialized only once** when they are compiled in a program.

Example: ILLUSTRATION OF WORKING OF static VARIABLES

```
void stat(void);
main ( )
{
    int i;
    for(i=1; i<=3; i++)
        stat( );
}
```

```
void stat(void)
{
    static int x = 0;

    x = x+1;
    printf("x = %d\n", x);
}
```

OUTPUT:

```
x = 1
x = 2
x = 3
```

3. External variables (Global Variables)

- Variables that are **active throughout the entire program** are called as **external variables (global variables)**.
- External variables are **declared outside the function body**. This storage class is created when variable is declared global.
- No memory is reserved for the variable. Variable retain the value throughout the execution of a program.
- This storage class can be accessed by any function in same or different program file and change its value.

Example: ILLUSTRATION OF PROPERTIES OF GLOBAL VARIABLES

```
int fun1(void);
int fun2(void);
int fun3(void);
int x ;                /* global */
main( )
{
    x = 10 ; /* global x */
    printf("x = %d\n", x);
    printf("x = %d\n", fun1());
    printf("x = %d\n", fun2());
    printf("x = %d\n", fun3());
}
fun1(void)
{
    x = x + 10 ;
}
int fun2(void)
{
    int x ;            /* local */
    x = 1 ;
    return (x);
}
```



```
fun3(void)
{
    x = x + 10 ;    /* global x */
}
```

OUTPUT:

```
x = 10
x = 20
x = 1
x = 30
```

External Declaration

- In the above program, the main cannot access the variable y as it has been declared after the main function. This problem can be solved by declaring the variable with the storage class named as “extern”

```
main()
{
    extern int y;    /*External declaration*/
    -----;
    -----;
}
func1()              /*External declaration*/
{
    extern int y;
}
int y;                /*Definition*/
```

4. Register Variables

- Register is a **special storage area** in a Central Processing Unit (CPU).
- There are **8 registers available** inside a Computer. Register variable can be accessed only by block in which it is declared. It cannot be accessed by any other function.
- Register variable declared using keyword **register**.

Example: ILLUSTRATION OF WORKING OF register VARIABLES

```
#include<stdio.h>
#include<conio.h>
void main()
{
    register int x;
    clrscr();
    for(x=1;x<=10;x++)
        printf(“%d”,x);
    getch();
}
```

2. Explain Recursion. Write a C program to find factorial of a number using recursion.
(May 2016)(May 2017)(Dec 2018)

A function that calls itself is known as a recursive function. And, this technique is known as recursion.

How recursion works?

```
void recurse()
{
    ... ..
    recurse();
    ... ..
}
int main()
{
    ... ..
    recurse();
    ... ..
}
```

How does recursion work?

```
void recurse()
{
    ... ..
    recurse();
    ... ..
}
int main()
{
    ... ..
    recurse();
    ... ..
}
```

The diagram shows a function `void recurse()` with a self-call `recurse();` and a `main()` function calling `recurse();`. Arrows indicate the flow of recursive calls and returns. A label 'recursive call' points to the self-call in `recurse()`.

Factorial of a Number Using Recursion

```
#include<stdio.h>
long int multiplyNumbers(int n);
int main() {
    int n;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
    printf("Factorial of %d = %ld", n, multiplyNumbers(n));
}
```

```
    return 0;
}
long int multiplyNumbers(int n) {
    if (n>=1)
        return n*multiplyNumbers(n-1);
    else
        return 1;
}
```

Output

Enter a positive integer: 6

Factorial of 6 = 720

3. Discuss branching and looping operations in C. (May 2014) (May 2018)

CONDITIONAL CONTROL STATEMENT

Those statements, which is **used to check the condition and based on the given condition, carry out specific operations.** Such type of statements is known as Conditional Control statement. Conditional Control statement is mainly of two main types.

They are

- (i) **Branching statements**
- (ii) **Looping statements**

BRANCHING STATEMENTS

This statement *executes the given condition based on the logical test. It performs two actions based on the result.*

The final result of the statement may be true or false. Some of the conditional execution statements are given below:

- Simple IF statement
- IF...ELSE statement
- Nested IF...ELSE statement
- IF ELSE IF LADDER statement
- Switch statement

SIMPLE IF STATEMENT

If statement is used in decision making execution. The condition part should not end with a semicolon.

The If structure has the following syntax

```
if(condition)
{
    Statement(s);
}
statement-x;
```

If statement executes if the given condition is true otherwise it does not produces any output.

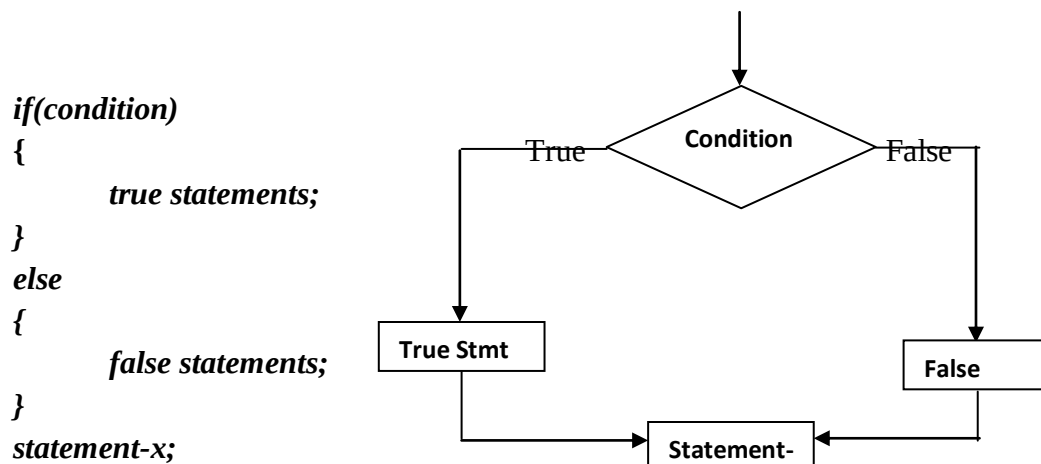
If the condition is true statement will be executed otherwise the statement is skipped to statement-x.

Calculate the absolute value of an integer

```
#include <stdio.h>           //Include the stdio.h file
void main ( )                // start of the program
{
    int number;               // Declare the variables
    printf("Type a number:"); // message to the user
    scanf("%d", &number);     // read the number from standard input
    if (number < 0)            // check whether the number is a negative
        number = - number;    // If it is negative then convert it into positive.
    printf("The absolute value is %d \n", number); // print the value
    getch();
}
```

IF ELSE STATEMENT

- If...else statement executes based on the given logical condition.
- **If the result of the condition is true, then program true statements is executed in the if block, otherwise program false statements will be executed in else block.**
- The syntax of if..else statement is as follows.



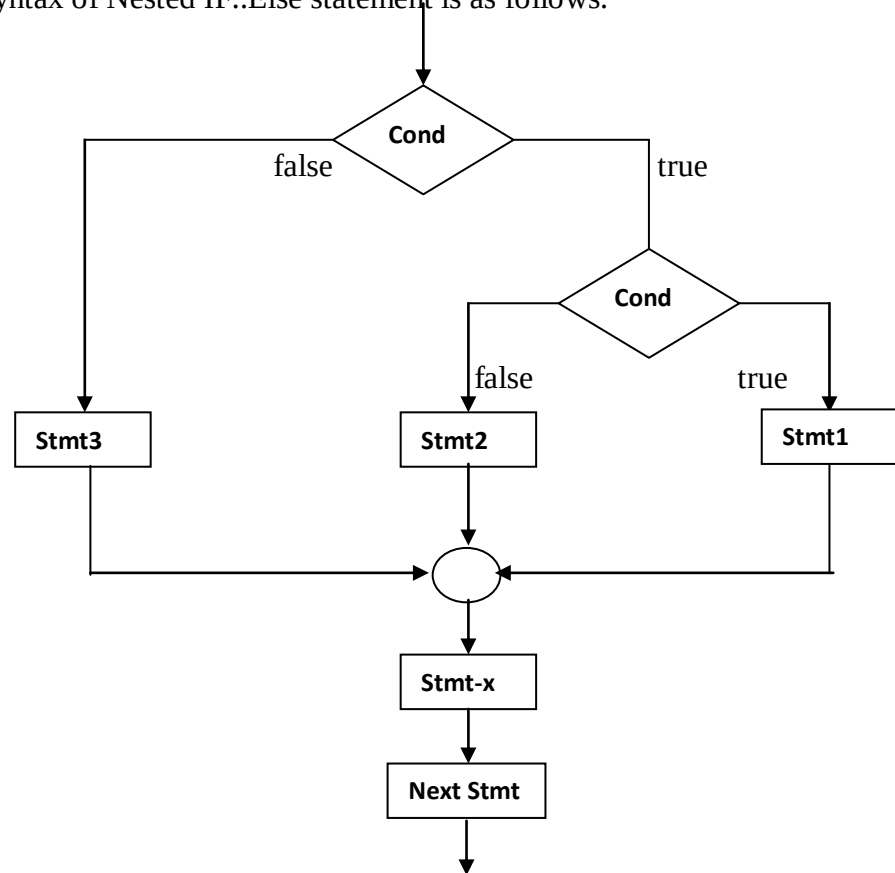
In these statement both if part and else part will not be executed at the same time. Either anyone of the both will be executed during run time.

NESTED IF...ELSE STATEMENT

- It is used to check with more than one Condition. If condition1 is true condition2 will be checked and statement1 will be executed followed by statement2 will be executed.
- If condition1 is false statement3 will be executed. Otherwise default statement will be executed. The syntax of Nested IF..Else statement is as follows.

```

if(condition1)
{
    if(condition2)
    {
        statement1;
    }
    else
    {
        statement2;
    }
}
else
{
    Statements3;
}
statement-x;
    
```

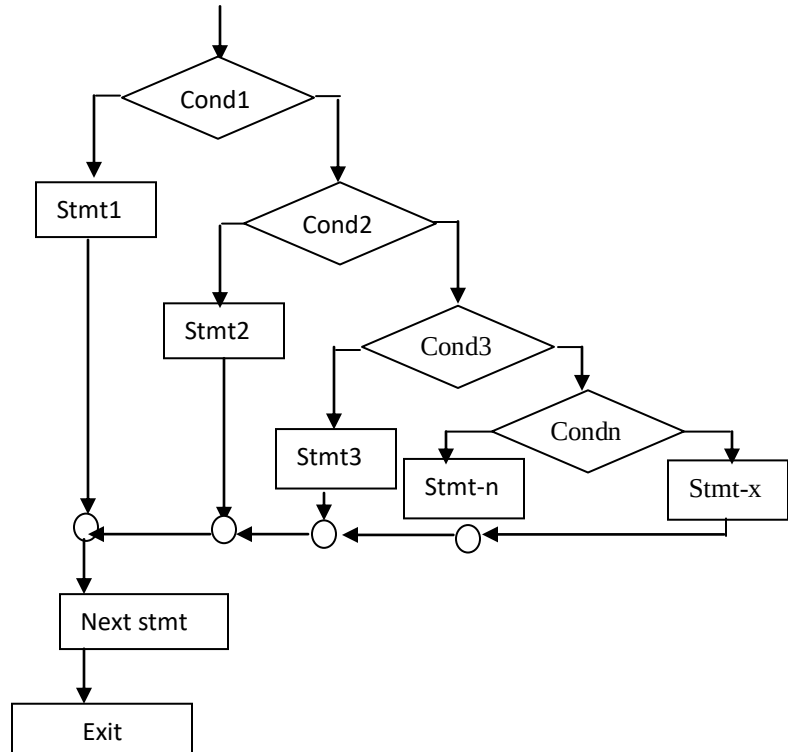


IF ELSE IF LADDER

- If multiple decisions are involved in if statement elseif ladder is used. Here condition1 is checked first. If it is true statement1 will be executed. Otherwise it checks for condition2.
- If condition2 is true statement2 will be executed. Otherwise it checks for condition-n and statement n will be executed.
- If all of the given condition becomes false then statement-x will be executed. The General form of elseif statement is as follows.

```

if(condition1)
    statement1;
elseif (condition2)
    statement2;
elseif (conditon3)
    statement3;
    .....
elseif (condition n)
    statement-n;
else
    statement-x;
    Next stmt
  
```



SWITCH CASE STATEMENT

It is also known as selection statement. It helps to make decision from number of cases. The syntax of Switch case statement is

```

switch(expression)
  
```

```

{
  
```

```

    case 1:
        statement;
        break;
  
```

```

    case 2:
        statement;
        break;
  
```

```

    .....
    default:
        statement;
  
```

```

}
  
```

LOOPING STATEMENTS

The process of executing a statement (or) group of statement repeatedly until a particular condition is satisfied is called Looping.

Loop is nothing but the part of a program code that execute repeatedly is called Loop. There are four types of Looping statements. They are

- (i) while loop
- (ii) do-while loop
- (iii) for loop
- (iv) nested for loop

WHILE LOOP

The general form of while statement is

```
while(test condition)
{
    statement(s);
}
```

while statement executes only if the given condition is true. While loop is also called **entry-controlled loop**. Entry-controlled loop also known as **pre-test**.

Example program for generating 'N' Natural numbers using while loop:

```
#include<stdio.h>           //include the stdio.h file
#include<conio.h>
void main()                 // Start of your program
{
    int n, i=1;              //Declare and initialize the variables
    clrscr();
    printf("Enter the upper limit number");           //Message to the user
    scanf("%d", &n);           //read and store the number
    while(i <= n)               // While statement with condition
    {                         // Body of the loop
        printf("%d\t",i);           // print the value of i
        i++;                       //increment I to the next natural number.
    }
    getch();
}
```

Output 1:

Enter the upper limit number 5

1 2 3 4 5

Output 2:

Enter the upper limit number -5

DO..WHILE LOOP

In do..while loop , statement is executed first then the condition is checked. If the given **condition is not true at-least one statement will be definitely executed**. If the condition is true then the while loop will be executed.

The syntax of the do while loop is:

```
do
{
    statement;
}
while(expression);
```

do-While loop is also called **exit-controlled loop**. Entry-controlled loop also known as **post-test**.

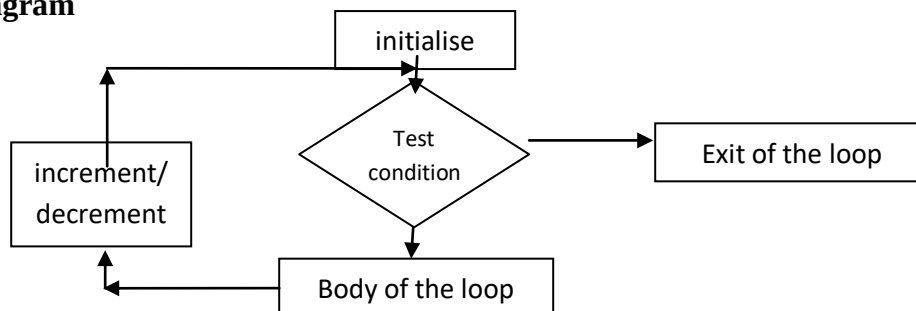
FOR LOOP

The for loop is an **Entry Controlled Loop**. It is a pre test.

The syntax of FOR loop is shown below:

```
for(initialization; test-condition; Increment/decrement)
{
    program statement(s);
}
```

Flow Diagram



Here initialization is done in the beginning of the for loop. Test condition is the place where the given condition is being checked. If the condition is true it executes body of the loop and then the value gets incremented/decremented, the body of the loop gets executed repeatedly until the condition became false. If the condition became false it exits the loop. Condition is checked every time using the increment/decrement value.

For loop example program:

```
/* The following is an example that finds sum of the first fifteen positive natural numbers*/
#include<stdio.h>                                //Include stdio.h file
```



```
#include<conio.h>
void main()                //start main program
{
    int i;
    clrscr();               //declare variable
    int sum=0,sumsq=0;       //declare and initialize variable.
    for(i=0;i<= 30; i+=2)   //for loop
    {
        sum+=i;             //add the value of I and store it to sum
        sumsq+=i*i;         //find the square value and add it to sumsq
    }                       //end of for loop
    printf("Sum of first 15 positive even numbers=%d\n",sum);    //Print sum
    printf("Sum of their squares=%d\n",sumsq);                  //print sumsq
    getch();
}
```

NESTED FOR LOOP

Loops containing more than one loops within itself is known as nested loop. Here outermost loop will start to execute first but the process of outermost loop ends only when the condition of innermost loop ends. Each loop must have a unique index variable. Each loop should be embedded with each other. Loops can have any number of exit point but should not permit entry into it. Loops should not overlap each other.

4. Explain the use of multidimensional arrays in C. (May 2014)

MULTI-DIMENSIONAL ARRAY

Multi-dimensional Array consists of (or) requires more than two square brackets and it may contain any number of values specified within square brackets. It may be three, four, five, six and so on. A Multi dimensional Array in general takes the following form.

data-type array-name[s1][s2].....[sn];

Data-type refers to a valid C data type. Array-name refers to a valid array name. s1,s2,s3, are sub scripts.

```
#include<stdio.h>
void main()
{
    int mark[2][4][3],i,j,k;
    int totmark,semsum,papsum;
    clrscr();
    for(i=0;i<2;i++)
```

```
{
printf("For the %d semester \n",i+1);
for(j=0;j<4;j++)
{
printf("\n Enter 3 marks for paper %d\n",j+1);
for(k=0;k<3;k++)
scanf("%d",&mark[i][j][k]);
}
}
totmark=0;
for(i=0;i<2;i++)
{
semsum=0;
printf("\n Semester %d\n",i+1);
printf("Paper marks \n");
for(j=0;j<4;j++)
{
papsum=0;
for(k=0;k<3;k++)

papsum+=mark[i][j][k];
printf("%d %d\n",j+1,papsum);
semsum+=papsum;
}
printf("Total marks in %d semester = %d \n",i+1,semsum);
totmark+=semsum;
}
printf("\n Grand total = %d",totmark);
getch();
}
```

INITIALIZING MULTI-DIMENSIONAL ARRAY

Multi-dimensional Array consists of (or) requires more than two square brackets and it may contain any number of values specified within square brackets. It may be three, four, five, six and so on. A Multi dimensional Array can be also initialized. The general form is

data-type array-name[s1][s2]...[sn] = { value1, value2, ... valuen};

Example

```
int num[2][3][2] = { 0,1,2,3,4,5,6,7,8,9,10,11};
```

Now here

```
num[0][0][0]=0;num[0][0][1]=1;num[0][1][0]=2;num[0][1][1]=3;num[0][2][0]=4;
```

```
num[0][2][1]=5;num[1][0][0]=6;num[1][0][1]=7;num[1][1][0]=8;num[1][1][1]=9;
num[1][2][0]=10;num[1][2][1]=11;
```

Example:

```
int table[2][3]={0,0,0,1,1,1};    ( or ) int table[2][3]={ {0,0,0},{1,1,1}}
int survey[3][5][12];              float table[5][4][5][3];
```

5. Write a C program to evaluate the following function to 0.0001 % accuracy: (Jan 2015)

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

Also , print the results in the tabular fashion for the values of X=0,30,45,60.

$\cos x = 1 - x^2/2! + x^4/4! - x^6/6! + \dots$

Solution:

```
/*.....cosx function.....*/
#include<stdio.h>
#include<math.h>
#include<conio.h>
double fact(double power)
{
    double f=1;
    int k;
    for(k=1;k<=power;k++)
        f=f*k;
    return(f);
}
void main()
{
    int i=1;
    double x,term,deno,lob,cos,power=2;
    clrscr();
    scanf("%lf",&x);
    term=1.0;
    cos=1.0;
    while(term>=0.0001)
    {
        lob=pow(x,power);
        deno=fact(power);
        term=lob/deno;
        power+=2;
        if(i%2==1)
            cos=cos-term;
        else
            cos=cos+term;
        i++;
    }
}
```

```
    }  
    printf("the result is= %lf",cos);  
getch();  
}
```

6. a. Write a program to find the sum of n numbers using recursion. (Jan 2016)

// Write a program to find the sum of n numbers using recursion

```
#include <stdio.h>  
int addNumbers(int n);  
int main() {  
    int num;  
    printf("Enter a positive integer: ");  
    scanf("%d", &num);  
    printf("Sum = %d", addNumbers(num));  
    return 0;  
}
```

```
int addNumbers(int n)  
{  
    if (n != 0)  
        return n + addNumbers(n - 1);  
    else  
        return n;  
}
```

Output

Enter a positive integer: 20

Sum = 210

b. explain in detail about the break, continue and goto statements with an example program.(Jan 2016)

UNCONDITIONAL CONTROL STATEMENT

The Execution of program can be altered by transferring the control from one place to another in a program without specifying the condition. This statement is known as Unconditional Control statement.

BREAK STATEMENT

Break statement is used to terminate the control from one place to another in a program. Break statement exits only single loop. Hence it is used together with the statements wherever necessary.

It is also used to exit from a loop. A break causes the innermost enclosing loop or switch to be exited immediately. It is also used together with while, do-while, for and switch statement.

/* A program to find the average of the marks*/

```
#include <stdio.h>           //include the stdio.h file to your program
#include<conio.h>
void main()                  // Start of the program
{
    int i, num=0;             //declare the variables and initialize
    float sum=0,average;      //declare the variables and initialize
    printf("Input the marks, -1 to end\n");    // Message to the user
    while(i)                  // While loop starts
    {
        scanf("%d",&i);       // read and store the input number
        if(i== -1)            // check whether input number is -1
            break;            //if number -1 is input skip the loop
        sum+=i;               //else add the value of I to sum
        num++;               // increment num value by 1
    }
    getch();
}
```

// End of the program

CONTINUE STATEMENT

It is used to continue the process. Here keywords continue is used together within a statement. This statement is specified after the condition. The continue statement causes the loop to be continued with the next iteration after skipping any statement in between.

```
#include <stdio.h>           //Include stdio.h file
void main()                  //start of the program
{
    int i=1, num, sum=0;      // declare and initialize the variables
    for (i = 0; i< 5; i++)    //for loop
    {
        printf("Enter the integer"); //Message to the user
        scanf("%d", &num);         //read and store the number
        if(num < 0)                 //check whether the number is less than zero
        {
```

```
        printf("You have entered a negative number"); // message to the user
        continue; // starts with the beginning of the loop
    } // end of for loop
    sum+=num; // add and store sum to num
}
printf("The sum of positive numbers entered = %d",sum); // print the sum.
} // end of the program.
```

GOTO Statement

GOTO statement is used to transfer the control from one part of the program to other ,with the use of label.

The syntax is given below:

goto label;

where **label** is an identifier.

Example program

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int age;
```

```
    clrscr();
```

```
    printf("Enter the Age :");
```

```
    scanf("%d",&age);
```

```
    if(age>=18)
```

```
    {
```

```
        goto m;
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("\n You are not Eligible to vote:");
```

```
        exit();
```

```
    }
```

```
    m: printf("You are Eligible to vote");
```

```
        getch();
```

```
    }
```

Output:

I/P and O/P

Enter the Age: 18

You are Eligible to vote

I/P and O/P

7. Write a program to sort a given set of numbers in ascending order.(5) (May 2016)*//C program to accept N numbers and arrange them in an ascending order*

```
#include <stdio.h>
void main()
{
    int i, j, a, n, number[30];
    printf("Enter the value of N \n");
    scanf("%d", &n);
    printf("Enter the numbers \n");
    for (i = 0; i < n; ++i)
        scanf("%d", &number[i]);
    for (i = 0; i < n; ++i)
    {
        for (j = i + 1; j < n; ++j)
        {
            if (number[i] > number[j])
            {
                a = number[i];
                number[i] = number[j];
                number[j] = a;
            }
        }
    }
    printf("The numbers arranged in ascending order are given below \n");
    for (i = 0; i < n; ++i)
        printf("%d\n", number[i]);
}
```

OUTPUT

Enter the value of N

6

Enter the numbers

3

78

90

456

780

200

The numbers arranged in ascending order are given below

3
78
90
200
456
780

Program Explanation

1. Declare an array of some fixed capacity, lets say 30.
2. From users, take a number N as input, which will indicate the number of elements in the array ($N \leq$ maximum capacity)
3. Iterating through for loops (from [0 to N)), take integers as input from user and print them. These input are the elements of the array.
4. Now, create a nested for loop with i and j as iterators.
5. Start the sorting in ascending order by extracting each element at position i of outer loop.
6. This element is being compared to every element from position i+1 to size-1 (means all elements present below this extracted element)
7. In case any of the extracted element is greater than the element below it, then these two interchange their position, else the loop continues.
8. After this nested loop gets executed, we get all the elements of the array sorted in ascending order.

8. A) Write a C program to find the addition and subtraction of two matrices. (6)(May 2016)

/*** MATRIX ADDITION *****/**

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int i,j,m,n;
```



```
int l[5][5],m[5][5],n[5][5];

clrscr();

printf("\n\t\t\t MATRIX ADDITION \n\n");

printf("\n Enter the Order of Matrix: \n");

scanf("%d%d",&m,&n);

printf("\nEnter the Elements of 1st Matris:\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
scanf("%d",&m[i][j]);
}
printf("\n");
}

printf("\n Enter the Elements of 2nd Matrix: \n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
scanf("%d",&n[i][j]);
}
printf("\n");
}

printf("\n The Resultant Matrix is: \n");

for(i=0;i<m;i++)
```

******* MATRIX SUBTRACTION *******

Page 26

```
for(j=0;j<n;j++)
{
scanf("%d",&m[i][j]);
}
printf("\n");
}
printf("\n Enter the Elements of 2nd Matrix: \n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
scanf("%d",&n[i][j]);
}
printf("\n");
}
printf("\n The Resultant Matrix is: \n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
l[i][j]=m[i][j]-n[i][j];
printf("%d\t",&c[i][j]);
}
printf("\n\n");
}
```

```
getch();  
  
}
```

B)Write a C program to concatenate two strings using built-in function.(5) (May 2016) (11)(NOV 2016)

```
// C program to concatenate two strings using built-in function  
#include <stdio.h>  
#include<string.h>  
int main()  
{  
    char s1[20]; // declaration of char array  
    char s2[20]; // declaration of char array  
    printf("Enter the first string : ");  
    scanf("%s", s1);  
    printf("\nEnter the second string :");  
    scanf("%s",s2);  
    strcat(s1,s2);  
    printf("The concatenated string is : %s",s1);  
    return 0;  
}
```

OUTPUT

```
Enter the first string : COMPUTER  
Enter the second string : PROGRAMMING  
The concatenated string is : COMPUTERPROGRAMMING
```

9. The Fibonacci numbers are defined as $F_n = F_{n-1} + F_{n-2}$, $F_1 = F_2 = 1$. Write a program to compute the Fibonacci number n. Do not use recursion. (Nov 2016).

```
#include<stdio.h>  
int main()  
{  
    int n1=0,n2=1,n3,i,number;  
    printf("Enter the number of elements:");  
    scanf("%d",&number);  
    printf("\n%d %d",n1,n2);//printing 0 and 1  
    for(i=2;i<number;++i)//loop starts from 2 because 0 and 1 are already printed  
    {  
        n3=n1+n2;  
        printf(" %d",n3);  
    }
```

```
n1=n2;
n2=n3;
}
return 0;
}
```

Output:

```
Enter the number of elements:15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

10. Write a C program to calculate the roots of the quadratic equation. (Dec 2017)

```
#include <math.h>
#include <stdio.h>
int main() {
    double a, b, c, discriminant, root1, root2, realPart, imagPart;
    printf("Enter coefficients a, b and c: ");
    scanf("%lf %lf %lf", &a, &b, &c);
    discriminant = b * b - 4 * a * c;

    // condition for real and different roots
    if (discriminant > 0) {
        root1 = (-b + sqrt(discriminant)) / (2 * a);
        root2 = (-b - sqrt(discriminant)) / (2 * a);
        printf("root1 = %.2lf and root2 = %.2lf", root1, root2);
    }

    // condition for real and equal roots
    else if (discriminant == 0) {
        root1 = root2 = -b / (2 * a);
        printf("root1 = root2 = %.2lf;", root1);
    }

    // if roots are not real
    else {
        realPart = -b / (2 * a);
        imagPart = sqrt(-discriminant) / (2 * a);

        printf("root1 = %.2lf+%.2lfi and root2 = %.2f-%.2fi", realPart, imagPart, realPart, imagPart);
    }
}
```

```
}  
return 0;  
}
```

OUTPUT

Enter coefficients a, b and c: 2.3

4

5.6

root1 = -0.87+1.30i and root2 = -0.87-1.30i

11. Write a C program to find maximum and minimum number of a given array. (Dec 2017)

```
#include <conio.h>  
int main()  
{  
    int a[1000],i,n,min,max;  
  
    printf("Enter size of the array : ");  
    scanf("%d",&n);  
  
    printf("Enter elements in array : ");  
    for(i=0; i<n; i++)  
    {  
        scanf("%d",&a[i]);  
    }  
  
    min=max=a[0];  
    for(i=1; i<n; i++)  
    {  
        if(min>a[i])  
            min=a[i];  
        if(max<a[i])  
            max=a[i];  
    }  
    printf("minimum of array is : %d",min);  
    printf("\nmaximum of array is : %d",max);  
  
    return 0;  
}
```

OUTPUT

Enter size of the array : 5
Enter elements in array : 2 6 3 1 9
minimum of array is : 1
maximum of array is : 9

12. Write a C program to count the letters in a sequence of characters. (May 2017)

```
#include <stdio.h>
#include <string.h>
int main()
{
    char string[100];
    int c = 0, count[26] = {0}, x;

    printf("Enter a string\n");
    gets(string);

    while (string[c] != '\0') {
        /** Considering characters from 'a' to 'z' only and ignoring others. */

        if (string[c] >= 'a' && string[c] <= 'z') {
            x = string[c] - 'a';
            count[x]++;
        }

        c++;
    }

    for (c = 0; c < 26; c++)
        printf("%c occurs %d times in the string.\n", c + 'a', count[c]);

    return 0;
}
```

OUTPUT

Enter a string
hello
e occurs 1 times in the string
h occurs 1 times in the string
l occurs 2 times in the string
o occurs 1 times in the string

13. Write a C program to calculate the average of 'n' numbers using the concept of arrays and function. (Sep 2020)

```
#include <stdio.h>
int main() {
    int n, i;
    float num[100], sum = 0.0, avg;
    printf("Enter the numbers of elements: ");
    scanf("%d", &n);
    while (n > 100 || n < 1) {
        printf("Error! number should in range of (1 to 100).\n");
        printf("Enter the number again: ");
        scanf("%d", &n);
    }
    for (i = 0; i < n; ++i) {
        printf("%d. Enter number: ", i + 1);
        scanf("%f", &num[i]);
        sum += num[i];
    }
    avg = sum / n;
    printf("Average = %.2f", avg);
    return 0;
}
```

Output

Enter the numbers of elements: 6

1. Enter number: 45.3

2. Enter number: 67.5

3. Enter number: -45.6

4. Enter number: 20.34

5. Enter number: 33

6. Enter number: 45.6

Average = 27.69

14. Write a menu driven C program to perform any five operations on string. (Sep 2020)

- i) Calculate Length of String
- ii) Reverse a given String
- iii) Concatenation of one string to another
- iv) Copy one String into another
- v) Compare two Strings

Program:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    char str1[20],str2[20];
    int ch,i,j;
    do
    {
        printf("\tMENU");
        printf("\n-----\n");
        printf("1:Find Length of String");
        printf("\n2:Find Reverse of String");
        printf("\n3:Concatenate Strings");
        printf("\n4:Copy String ");
        printf("\n5:Compare Strings");
        printf("\n6:Exit");
        printf("\n-----\n");
        printf("\nEnter your choice: ");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("Enter String: ");
                scanf("%s",str1);
                i=strlen(str1);
                printf("Length of String : %d\n\n",i);
                break;
            case 2:
                printf("Enter String: ");
                scanf("%s",str1);
                //strrev(str1);
                printf("Reverse string : %s\n\n",str1);
                break;
            case 3:
                printf("\nEnter First String: ");
                scanf("%s",str1);
```

```
    printf("Enter Second string: ");
    scanf("%s",str2);
    strcat(str1,str2);
    printf("String After Concatenation : %s\n\n",str1);
break;
case 4:
    printf("Enter a String1: ");
    scanf("%s",str1);
    printf("Enter a String2: ");
    scanf("%s",str2);
    printf("\nString Before Copied:\nString1=\"%s\",String2=\"%s\"\n",str1,str2);
    strcpy(str2,str1);
    printf("-----\n");
    printf("\nWe are copying string String1 to String2\n\n");
    printf("-----\n");
    printf("String After Copied:\nString1=\"%s\", String2=\"%s\"\n\n",str1,str2);
break;
case 5:
    printf("Enter First String: ");
    scanf("%s",str1);
    printf("Enter Second String: ");
    scanf("%s",str2);
    j=strcmp(str1,str2);
    if(j==0)
    {
        printf("Strings are Same\n\n");
    }
    else
    {
        printf("Strings are Not Same\n\n");
    }
break;
case 6:
    exit(0);
break;
default:
    printf("Invalid Input. Please Enter valid Input.\n\n ");
}
}while(ch!=6);
```

```
    return 0;  
}
```

Output:

MENU

- 1: Find length of String**
- 2: Find Reverse of String**
- 3: Concatenate Strings**
- 4: Copy String**
- 5: Compare Strings**
- 6: Exit**

Enter your choice : 3

Enter First String : Tutorial

Enter Second String: Ride

String After Concatenation : TutotiralRide

MENU

- 1: Find length of String**
- 2: Find Reverse of String**
- 3: Concatenate Strings**
- 4: Copy String**
- 5: Compare Strings**
- 6: Exit**

Enter your choice : 6